

Consolidated Interview Findings

Next-Generation Trading Platform MVP

This document summarises the points raised in the interviews. It deliberately separates:

1. **Confirmed interview findings** — items explicitly mentioned by interviewees or facilitators.
2. **Interpretations and team decisions required** — areas implied by the interviews but not yet agreed as detailed requirements.
3. **Architectural considerations added by the architect** — sensible design questions, but not requirements stated during the interviews.
4. **Future roadmap items** — enhancements explicitly suggested for later phases.

This distinction is important: the MVP should be built primarily around confirmed stakeholder needs, while unconfirmed design choices should be documented transparently as team assumptions or future decisions.

1. Confirmed Interview Findings

1.1 Scope and Asset-Class Priority

The system should initially focus on **equities**. The interviews consistently recommended building one asset class properly before expanding further.

- Equities should be the first priority.
- Bonds are relevant to the broader platform vision, but should follow once the equity foundation is stable.
- Derivatives should only be considered if time permits.
- Different asset classes may need different metrics, strategies, and potentially different screens or views.
- The platform should be designed with an extensible foundation, rather than attempting to support every asset class immediately.

The goal is to demonstrate a useful and stable MVP, not to implement full multi-asset-class coverage.

1.2 Order Execution

The platform should support multiple order types. The following examples were explicitly mentioned:

- Limit Order
- Stop Order
- Fill-or-Kill Order

Users should be able to select an order type. The system should then execute or respond in accordance with the conditions of that order type.

The interviews did not specify all detailed order-lifecycle rules. However, they clearly established that order execution must be more than a simple buy/sell button: it should reflect the conditions associated with the selected order type.

1.3 Holdings, Portfolio Management, and Post-Trade Information

After each trade, Traders should be able to view useful information about their current situation.

Examples explicitly mentioned include:

- Current positions
- Remaining trading limit
- Portfolio risk
- Position size
- Profit and loss
- Exposure

- Overall portfolio-risk indicators

The existing report was understood to be a **Holdings Report**, rather than a post-trade execution report. It should show the user's current portfolio positions.

Relevant holdings information includes:

- Entity, ticker, or instrument
- Quantity
- Average Cost
- Last Price
- Current holding value
- Profit and loss
- Profit and loss percentage

The label "**Market Value**" may be confusing because it could be interpreted as the current market price of an instrument rather than the total value of a position. More explicit alternatives suggested in the interviews include:

- Position Value
- Current Position Value
- Total Market Value
- Holding Value
- Position Market Value

At the portfolio level, users should be able to understand:

- Total portfolio value
- Total invested amount or total cost
- Overall performance in absolute terms
- Overall performance in percentage terms
- Which holdings are making money
- Which holdings are losing money
- Which holdings may require further analysis

The following minimum portfolio metrics were recommended:

- Total Portfolio Value
- Total Cost / Total Invested Amount
- Total Unrealised P&L
- Total Unrealised P&L %
- Number of Holdings
- Cash Balance, if applicable

The following holdings-table fields were recommended:

- Ticker
- Company or Instrument Name
- Quantity
- Average Cost
- Last Price
- Total Cost
- Current Position Value
- Unrealised P&L
- Unrealised P&L %
- Weight in Portfolio %

The interviews also explicitly recommended the following calculations:

- **Position Value = Quantity × Last Price**
- **Total Cost = Quantity × Average Cost**
- **Unrealised P&L = Position Value – Total Cost**
- **Unrealised P&L % = Unrealised P&L / Total Cost**
- **Portfolio Value = Sum of all Position Values**
- **Total Portfolio P&L = Sum of all Unrealised P&L**

For the demonstration, approximately five or six tickers are sufficient. The team should show a range of capabilities rather than repeat the same scenario across many instruments.

1.4 User Experience, Reporting, and Visualisation

The current user experience should be improved.

Interview feedback highlighted the following issues:

- Users currently need to scroll to find information.
- Important information should be displayed near the top of the page.
- The main page should provide a quick snapshot of the user's situation.
- Users should be able to click from summary information into detailed views.
- The interface currently feels crowded.
- Some text and content are too small and difficult to read.
- Users should be able to find, click, and access information easily.
- Favourite or frequently used items should be easier to access.

If time allows, possible useful visualisations include:

- Portfolio value chart
- P&L chart
- Allocation pie chart
- Top gainers and losers
- Sector or asset-allocation chart

The exact "top five" numbers or metrics for a Trader's main screen had not yet been confirmed. Further input from Traders is required.

1.5 Risk Management and Access Management

Risk management is an important expected capability. Traders should be able to understand how much risk they are taking and what trading capacity remains.

Possible information mentioned includes:

- Position size
- Remaining trading limit
- P&L
- Exposure
- Portfolio risk
- Overall portfolio-risk indicators

The exact risk metrics required by Traders still require further research.

The platform should not be available to unauthorised users. Access control and segregation of duties should be considered.

The interviews discussed different permissions or views for:

- Traders
- Risk-management users
- Administration functions or Administrators

A preferred UX principle is role-based visibility:

- Users should see functions relevant to their role.
- If a user does not have permission to access a feature, that feature should ideally be hidden rather than shown and rejected after selection.

However, implementing complete role-based navigation should not take priority over core functionality or demo stability. If incomplete, it can be included in the future roadmap.

1.6 GenAI, Technical Support, and Market Data

GenAI may be used as an advisory and supportive capability. Suggested uses include:

- Summarising news

- Supporting analysis
- Providing explanations
- Helping users understand data
- Generating insights or reports
- Supporting technical analytics or user assistance

A key guardrail was explicitly confirmed:

GenAI must not make trading decisions or autonomously execute trades. The user remains responsible for the final trading decision.

The team may use GenAI during development, including prompt refinement, implementation exploration, and debugging. However, the team must review generated output critically and remain responsible for all final design choices.

Training a machine-learning model using the labelled news dataset is optional and not mandatory.

The supplied minute-level simulated market data may be replayed at an accelerated pace for demonstration purposes. This should be explained clearly during the demo.

A free external market-data source may be added if time permits, but it is not mandatory and should not distract from stabilising the core product.

1.7 Development, Delivery, and Presentation

The facilitator supported an Agile approach rather than Waterfall.

The team should maintain records of:

- Decisions made
- Reasons for decisions
- Problems encountered
- Solutions adopted
- Feedback received
- Changes made after feedback
- Ideas considered but not implemented

The team should demonstrate collective ownership in the final presentation. Members should generally present using “we” and “the team,” rather than focusing excessively on individual contributions.

Development priorities were clearly stated:

1. Fix obvious bugs and error messages.
2. Stabilise the main demo flow.
3. Ensure core functionality works correctly.
4. Improve the usability and layout of key screens.
5. Add enhancements only if time remains.
6. Move unfinished ideas into the future roadmap.

The final presentation should tell a story rather than simply demonstrate screens. It should cover:

- The original problem and objective
- The team’s understanding of the requirements
- Technology choices and rationale
- Architecture or system-flow diagrams
- What was built
- Challenges encountered
- How challenges were resolved
- Stakeholder feedback and resulting changes
- Future enhancements

A live demo can be impressive but is risky. A recorded demo, narrated live by a team member, is an acceptable and safer alternative. The team should rehearse together and ensure transitions between presenters are smooth.

2. Interpretations and Team Decisions Required

The following points are implied by the interviews or require team decisions, but they were not agreed as detailed stakeholder requirements.

2.1 Scope Decisions

- Confirm which order types will be included in the MVP.
- Confirm the MVP definition of “Remaining Trading Limit.”
- Confirm the most important Trader dashboard metrics.
- Confirm which risk metrics are practical and useful for the MVP.
- Confirm which charts, if any, can be included within the available time.
- Confirm whether Cash Balance will be included in the portfolio summary.
- Confirm the final set of user roles and permissions for the MVP.
- Confirm the specific GenAI use case to implement.
- Confirm which technical-analysis features or indicators, if any, will be included.
- Confirm whether external market-data integration will be attempted.

2.2 Suggested MVP Prioritisation

Based on the interview guidance, the team should prioritise:

- Equity trading functionality
- Stable buy/sell or order-entry flow
- Holdings report
- Portfolio summary
- P&L calculations
- Market-data display
- Clear reporting and charting where feasible
- A simple risk view
- A stable, rehearsed demo flow

Any functionality that cannot be completed reliably should be documented as a roadmap item rather than presented as partially complete core functionality.

3. Architectural Considerations Added by the Architect

Not Confirmed Interview Requirements

The following are sensible system-design considerations, but they were **not explicitly requested or defined in the interviews**. They should therefore be treated as team design questions, assumptions, or possible future enhancements—not as confirmed stakeholder requirements.

3.1 Order-Lifecycle Design Questions

- How should each order type behave when simulated prices change?
- What should trigger a Stop Order?
- Should a triggered Stop Order become a Market Order or a Limit Order?
- Should partial fills be supported?
- Should unfilled orders expire, remain open, or be cancelled?
- Should order amendment and cancellation be supported?
- Should commissions, spreads, or slippage be included in calculations?

3.2 Risk-Control Design Questions

- What specific pre-trade validation should be implemented?
- Should the system check available cash before accepting an order?
- Should there be maximum order quantity, order value, or position-size limits?
- How should a risk-limit breach be handled: warning, rejection, approval, or simulated acceptance?
- Should the system calculate gross exposure, net exposure, concentration, volatility, or drawdown?
- Should short selling, leverage, restricted instruments, and trading-hour rules be

considered?

3.3 Authentication, Role, and Audit Design Questions

- What authentication mechanism should be used for the MVP?
- Should there be a simple demonstration login or a more formal identity approach?
- What role combinations should be permitted or prohibited?
- Should users be allowed to approve their own risk-breaching orders?
- Which events should be audit-logged?
- How should audit logs be retained, protected, and reviewed?

3.4 Technical Architecture and Operations Questions

- Should the application be a modular monolith or have separate frontend and backend components?
- Should it expose APIs for future integration?
- What database and persistence model should be used?
- Which data should persist after application restart?
- Should deployment, monitoring, health checks, automated testing, and CI/CD be included in the MVP?
- How should external and simulated market data be normalised if both are used?

These questions are relevant to a production-grade architecture, but the interviews clearly advised that the team should not sacrifice stable MVP functionality for unnecessary complexity.

4. Confirmed Future Roadmap Themes

The interviews explicitly suggested that incomplete but valuable capabilities can be presented as future enhancements.

Possible roadmap themes include:

- More refined role-based access control and navigation
- Improved administration functions
- Better risk-management functionality
- Failed-trade and exception handling
- More comprehensive validation
- Clearer user-facing error messages
- Error logging
- External live market-data integration
- More sophisticated technical analytics
- More complete portfolio-management features
- Production-grade low-latency architecture
- Stronger operational controls
- Advanced monitoring and logging

For a future production platform, lower-latency technologies and infrastructure may be required, potentially including C++, Java, real-time environments, specialised low-latency infrastructure, and, in some use cases, FPGA-based components.